

Documentação — Tech do Bem — Sprint 4 IA

Disciplina: Artificial Intelligence e Chatbot

Curso: 1TDSR — FIAP

Equipe:

RM	Nome
RM568542	Hugo Souza de Jesus
RM566815	Lucas Campanhã dos Santos
RM567010	Lucas Marcelino Pompeu

Sumário

1. Objetivo e escopo do projeto
 2. Dataset — geração, features e variável-alvo
 3. Pipeline de pré-processamento e treino
 4. Seleção de modelo — comparação 5-fold
 5. Métricas no conjunto de teste
 6. Serialização do modelo (.joblib)
 7. Deploy — API Flask
 8. Demonstração — consumo pela página web
 9. Como executar (passo a passo reproduzível)
10. Atendimento à rubrica

1. Objetivo e escopo do projeto

A **Tech do Bem** é uma ONG que oferta atendimento odontológico gratuito a populações em vulnerabilidade. Para apoiar a tomada de decisão sobre onde alocar campanhas e dentistas, foi treinado um modelo preditivo que classifica a **demandas odontológica** de cada bairro de São Paulo em três níveis (**Baixa**, **Media**, **Alta**) a partir de indicadores socioeconômicos.

O modelo é exposto como **API HTTP** (**/predict**, **/predict/batch**, **/health**) e consumido pelo frontend em duas funcionalidades:

- **Mapa de calor** dos bairros, com cor proporcional à classe predita.
 - **Página dinâmica** **/colaborador/area/:bairro** que mostra a predição com barras de probabilidade por classe e os pacientes daquele bairro.

A predição em tempo real ajuda coordenadores a priorizar regiões com maior necessidade.

2. Dataset — geração, features e variável-alvo

2.1 Origem dos dados

Por restrições éticas (dados de saúde + endereço de pacientes reais), o dataset é **sintético**, gerado de forma reproduzível por `data/gerar_dataset.py` com `seed=42`. A geração é livre de IA generativa — todo o código está versionado e auditável.

- **2050 linhas** (média de ~25 amostras por bairro).
 - **82 bairros reais** de São Paulo, com coordenadas geográficas verdadeiras.
 - Label balanceado por quantilização do score latente (terços).

2.2 Features (variáveis explicativas)

Feature	Tipo	Descrição
<code>densidade_pop</code>	float	Densidade populacional (hab/km ²) — gerada por bairro com ruído.
<code>idh</code>	float	Índice de Desenvolvimento Humano local (0 – 1).
<code>dist_ubs_km</code>	float	Distância média até a UBS mais próxima (km).
<code>historico_atendimentos</code>	int	Número de atendimentos anteriores no histórico.
<code>pct_exames_pendentes</code>	float	Percentual de exames pendentes (0 – 1).
<code>populacao_infantil_pct</code>	float	Percentual de população ≤ 14 anos (0 – 1).
<code>dia_semana_cadastro</code>	int	Dia da semana (0 – 6) — decoy (ruído puro).

A `dia_semana_cadastro` é **proposital**: como ela não influencia o label, o modelo deve aprender a ignorá-la — o que serve como teste de sanidade da feature importance.

2.3 Variável-alvo

`demanda` ∈ {`Baixa`, `Media`, `Alta`} — gerada de forma controlada:

1. **Score latente** com ruído gaussiano:

2. $s = 0.40 * (1 - idh_norm)$
3. $+ 0.25 * dist_norm$
4. $+ 0.20 * pct_exames_norm$
5. $+ 0.15 * densidade_norm$
6. $+ N(0, 0.12)$

2. **Discretização por quantis** em 3 terços iguais → **Baixa / Media / Alta**.

O ruído gaussiano ($\sigma=0.12$) **impede acurácia perfeita** — comprova que o modelo está aprendendo o padrão real e não decorando o rótulo.

2.4 Justificativa da escolha

- Modelar o problema como **classificação multi-classe** (em vez de regressão) permite ações operacionais claras: "este bairro precisa de mais campanhas" em vez de "este bairro tem score 0.62".
 - O dataset sintético com 7 features cobre dimensões reconhecidamente correlacionadas à demanda de saúde pública (IDH, distância à UBS, exames pendentes), permitindo storytelling pedagógico.

3. Pipeline de pré-processamento e treino

Implementada em [modelo/train.py](#) e replicada no notebook [notebooks/01_pipeline_demanda.ipynb](#).

3.1 Codificação da variável-alvo

LabelEncoder mapeia as três classes textuais para inteiros (0, 1, 2). O encoder é serializado junto com o pipeline para garantir o mapeamento reverso na API.

3.2 Normalização (condicional)

A normalização (**StandardScaler**) só é aplicada para o modelo linear — os modelos baseados em árvores (Random Forest, XGBoost) não exigem normalização e a inclusão é dispensável:

Modelo	Normalização
RandomForest	Não (insensível à escala)
XGBoost	Não (insensível à escala)
LogisticRegression	Sim — StandardScaler no Pipeline

3.3 Split treino/teste

`train_test_split(test_size=0.20, stratify=y, random_state=42):`

- **1640 linhas** para treino.
 - **410 linhas** para teste.
 - Estratificação garante distribuição equilibrada das 3 classes em ambas as partições.

3.4 Engenharia de features

A `dia_semana_cadastro` é mantida intencionalmente como decoy — a análise de feature importance pós-treino confirma que o modelo aprendeu a ignorá-la (ela cai fora do top-5).

4. Seleção de modelo — comparação 5-fold

Três modelos competiram entre si, comparados por `f1_macro` (média e desvio) em **validação cruzada estratificada de 5 folds**:

Modelo	f1_macro (CV)	Desvio	Observação
<code>random_forest</code>	0.567	± 0.018	Baseline robusto sem normalização
<code>xgboost</code>	0.537	± 0.027	Referência tabular forte
<code>logistic_regression</code>	0.587	± 0.024	Vencedor — controle linear com StandardScaler

Por que LogisticRegression venceu? O score latente do dataset é uma **combinação linear** de 4 features normalizadas + ruído gaussiano. Esse é o cenário ideal para regressão logística com normalização — modelos mais expressivos (RF, XGB) tendem a *overfittar* o ruído. É um resultado coerente com a teoria, não um acaso.

4.1 Fallback de robustez

Se a importação do `xgboost` falhar (ambientes restritivos), o `train.py` substitui automaticamente por `GradientBoostingClassifier` do scikit-learn — mantendo 3 modelos comparados sem alteração do código.

5. Métricas no conjunto de teste

Avaliação do modelo vencedor (`logistic_regression`) na partição de teste (410 linhas):

Métrica	Valor
Accuracy	0.639
F1-macro	0.631
F1-macro (CV 5-fold)	0.587
Linhas de treino / teste	1640 / 410

5.1 Matriz de confusão

		predito		
		Alta	Baixa	Media
verdadeiro	Alta	102	10	25
	Baixa	12	101	24
	Media	38	39	59

- A classe **Media** é a mais difícil — sobrepõe-se com **Baixa** e **Alta** nos extremos do quantil — comportamento esperado.
 - **Baixa** e **Alta** têm precisão acima de 85% — adequado para uso operacional.

5.2 Top-5 feature importances (coeficientes da regressão logística)

```
| Rank | Feature | Importância (|coef|) |
|---|---|---|
| 1 | idh | 0.632 |
| 2 | pct_exames_pendentes | 0.340 |
| 3 | dist_ubs_km | 0.331 |
| 4 | densidade_pop | 0.206 |
| 5 | historico_atendimentos | 0.068 |
```

A **dia_semana_cadastro** (decoy) **ficou fora do top-5** — prova de que o modelo está aprendendo o sinal correto e não memorizando ruído.

6. Serialização do modelo (.joblib)

`modelo/train.py` serializa um **único artefato** em `modelo/modelo_demanda.joblib` contendo:

```
{
  "pipeline":      <sklearn.Pipeline ajustado>,      #
incluindo StandardScaler
  "label_encoder": <LabelEncoder ajustado>,
  "features":      ["densidade_pop", "idh", ...],
  "modelo_nome":   "logistic_regression",
  "versao":        "logistic_regression-v1",
```

```
}
```

Esse formato permite que a API faça **uma única chamada** `joblib.load()` no startup para ter o serviço inteiro carregado — atendendo ao requisito da rubrica de "serviço inteiro contido dentro de um único objeto que possa ser facilmente copiado para outras máquinas".

7. Deploy — API Flask

Implementada em `api/app.py` com `flask-cors`. Em produção roda via **gunicorn** com 2 workers (`Dockerfile`).

7.1 Endpoints

GET /health

Verificação rápida do estado do serviço. Resposta:

```
{
  "status": "ok",
  "modelo_carregado": true,
  "versao": "logistic_regression-v1",
  "bairros_disponiveis": 82
}
```

POST /predict — modo "features completas"

Recebe os 7 valores de features em JSON e retorna a predição com probabilidades:

```
curl -X POST http://localhost:5001/predict \
  -H 'Content-Type: application/json' \
  -d '{
    "densidade_pop": 12000,
    "idh": 0.78,
    "dist_ubs_km": 1.5,
    "historico_atendimentos": 45,
    "pct_exames_pendentes": 0.3,
    "populacao_infantil_pct": 0.22,
    "dia_semana_cadastro": 3
  }'
```

```
{
  "classe": "Baixa",
```

```
"modelo-versao": "logistic_regression-v1",
"probabilidades": {"Alta": 0.069, "Baixa": 0.708, "Media":
0.223}
}
```

POST /predict — modo "lookup por bairro"

Conveniência para o frontend: agrega as features médias do bairro a partir do CSV e dispara a predição. Retorna 404 se o bairro não existir.

```
curl -X POST http://localhost:5001/predict \
-H 'Content-Type: application/json' \
-d '{"bairro": "Brás"}
```

```
{
  "bairro": "Brás",
  "classe": "Alta",
  "modelo-versao": "logistic_regression-v1",
  "probabilidades": {"Alta": 0.667, "Baixa": 0.055, "Media":
0.278}
}
```

POST /predict/batch

Versão em lote (array de inputs). Cada item da resposta tem **index** correspondente ao input — evita N chamadas pelo frontend quando renderizando o mapa de calor.

7.2 Códigos HTTP

Status	Significado
200	Predição realizada com sucesso
400	JSON ausente, malformado ou features fora do range esperado
404	Bairro não encontrado em data/demanda_bairros.csv
405	Método HTTP incorreto (ex.: GET /predict)
500	Falha inesperada na predição

7.3 CORS

Whitelist explícita de origens (sem *):

- <http://localhost:5173> (frontend Vite local)
 - <https://techdobem.labs-lcs.com> (deploy próprio do grupo)
 - <https://techdobem.vercel.app> (deploy Vercel)
-

8. Demonstração — consumo pela página web

A página [/colaborador/area/:bairro](#) no frontend consome [POST /predict](#) com [{bairro: ...}](#) ao montar. A resposta alimenta:

- **Hero card** com a classe predita.
 - **Barras de probabilidade** por classe ([Baixa](#) / [Media](#) / [Alta](#)).
 - **Identificação do modelo** ([logistic_regression-v1](#)).
 - **Fallback gracioso**: se a API estiver fora, exibe um banner "Predição indisponível" sem quebrar a listagem de pacientes.

A view de mapa também consome [/ranking](#) (variante de [/predict/batch](#)) para colorir cada bairro proporcionalmente à classe.

Screenshots reais da página web estão disponíveis na entrega oficial em PDF (gerada via Print → PDF deste documento).

9. Como executar (passo a passo reproduzível)

9.1 Pré-requisitos

- **Python 3.12** (ambiente alvo do [requirements.txt](#)). Em Python 3.9 use versões mais antigas: [numpy<2](#) [pandas<2.2](#) [scikit-learn<1.5](#) [xgboost<2.1](#).
 - **pip** atualizado.
 - **(opcional) Docker** para deploy via container.

9.2 Execução local — do zero

```
git clone https://github.com/Tech-do-Bem-FIAP/Backend_AI.git
cd Backend_AI
```

```
# 1) ambiente isolado
python -m venv .venv
source .venv/bin/activate          #
Windows: .venv\Scripts\activate
```

```
# 2) dependências
pip install -r requirements.txt
```

```
# 3) (opcional) regerar dataset — já vem commitado
python data/gerar_dataset.py
```



```
# 4) treinar - gera modelo/modelo_demanda.joblib + modelo/
metrics.json
python modelo/train.py
```

```
# 5) subir a API
python -m flask --app api.app run --port 5001
```

Em outra janela:

```
curl http://localhost:5001/health
curl -X POST http://localhost:5001/predict \
  -H 'Content-Type: application/json' \
  -d '{"bairro":"Brás"}
```

9.3 Execução via Docker

```
docker build -t backend-ai .
docker run -p 5001:5001 backend-ai
curl http://localhost:5001/health
```

A imagem ([Dockerfile](#)) usa [python:3.12-slim](#), copia somente [api/](#), [modelo/](#), [modelo_demanda.joblib](#), [data/demanda_bairros.csv](#) e roda [gunicorn -w 2 -b 0.0.0.0:5001 api.app:app](#).

9.4 Notebook (EDA + matriz de confusão)

```
pip install jupyter
jupyter notebook notebooks/01_pipeline_demanda.ipynb
```

O notebook contém os 5 gráficos exigidos pela Sprint 3 (histograma de IDH, scatter idh x dist_ubs colorido por classe, heatmap de correlação, barras de feature importance, matriz de confusão) com comentários interpretativos sobre cada um.

9.5 Troubleshooting

Sintoma	Causa	Solução
Address already in use na porta 5000	macOS AirPlay Receiver	Usar --port 5001 (padrão do projeto)
Erro de import do xgboost	Wheel indisponível na plataforma	O train.py já tem fallback para GradientBoostingClassifier

Bairro não encontrado no /predict	String com acento desencodada	Verificar Content-Type: application/json e UTF-8
ModuleNotFoundError: api.app ao rodar flask	venv não ativado	source .venv/bin/activate

10. Atendimento à rubrica

Critério Sprint 4 (Artificial Intelligence e Chatbot)	Pontos	Status	Evidência
Pipeline de pré-processamento (.ipynb)	40	✓	notebooks/01_pipeline_de_manda.ipynb + modelo/train.py — LabelEncoder, StandardScaler condicional, split estratificado, engenharia de features (decoy)
Treinamento de ≥ 3 modelos diferentes	(incluído nos 40)	✓	RandomForest, XGBoost, LogisticRegression (com fallback GradientBoosting se XGB indisponível)
Seleção do melhor modelo	(incluído nos 40)	✓	f1_macro em 5-fold estratificado — vencedor: LogisticRegression (0.587)
Serialização em .joblib	20 (parte dos 40 de deploy)	✓	modelo/modelo_demand_a.joblib — pipeline + LabelEncoder + features em um único artefato

API com <code>/predict</code>	10 (parte dos 40 de deploy)	✓	<code>api/app.py</code> aceita features completas OU <code>{bairro}</code>
API com <code>/health</code>	10 (parte dos 40 de deploy)	✓	<code>api/app.py</code> retorna status, versão, bairros disponíveis
Documento com demonstrações	20	✓	Este <code>DOCUMENTACAO.md</code> (exportar Print → PDF) — instruções reproduzíveis em §9
Total	100		Pontuação máxima esperada

10.1 Mitigação de riscos pedagógicos

Risco	Mitigação
"O modelo não está realmente aprendendo, só decorou."	Feature decoy (<code>dia_semana_cadastro</code>) ficou fora do top-5 — evidência objetiva de aprendizado correto.
"A acurácia está baixa demais."	Limite teórico devido ao ruído gaussiano $\sigma=0.12$ injetado no score latente. F1-macro 0.63 é compatível e desejável (não há overfitting).
"Por que LogReg ganhou de XGBoost?"	O score latente é linear em 4 features normalizadas. Modelos mais expressivos overfittam o ruído — resultado coerente, não ocasional.
"Como reproduzir em outra máquina?"	Tudo determinístico via <code>random_state=42</code> + <code>requirements.txt</code> pinado + Docker. §9 lista passo a passo testado.

10.2 Repositórios irmãos do projeto Tech do Bem

- `Tech-do-Bem-FIAP/Backend_AI` — este repositório

- [Tech-do-Bem-FIAP/Backend_Java](#) — API Java Quarkus
- [Tech-do-Bem-FIAP/Frontend](#) — SPA React + Vite
- [Tech-do-Bem-FIAP/Database](#) — schema e seed Oracle (Sprint 4 Building Relational Database)